

Module Self-Healing

AI Test Automation Framework

Documentation detaillee pour la soutenance

Rapporteur et President - Experts en automatisation et IA

Projet NoveoCare

Mariam El Houche-Ext

Table des matieres

1. Architecture du module self-healing

- 1.1 Principe general
- 1.2 Les 8 classes du package healing

2. Pipeline de healing complet

- 2.1 Capture de snapshot
- 2.2 Resolution de locator (cache + API + fallback)
- 2.3 Decorateurs WebDriver et WebElement

3. Description detaillee des classes

- 3.1 HealingLocatorResolver
- 3.2 HealingWebDriver
- 3.3 HealingWebElement
- 3.4 SelfHealingClient
- 3.5 HealingBaseline
- 3.6 HealingRequest et HealingResponse
- 3.7 ElementInfo

4. Utilitaires (HealingUtils)

- 4.1 Conversion By -> Map et Map -> By
- 4.2 Capture DOM et debug
- 4.3 Inference de type et coordonnees

5. Integration Dashboard (DashboardReporter)

- 5.1 Push des resultats Cucumber
- 5.2 Push des evenements de healing

6. Generation de features via Colab (ColabClient)

- 6.1 Principe et flux
- 6.2 Appel API et extraction JSON

7. Les 3 modes de guerison detailles

- 7.1 Baseline Java
- 7.2 CI Healing Stub
- 7.3 Colab (IA complet)
- 7.4 Tableau comparatif

8. Flux de donnees healing vers dashboard

- 8.1 Diagramme de flux
- 8.2 Payload des evenements
- 8.3 Fallback 3 niveaux des metriques

9. Q&A - Questions possibles

- 9.1 Questions sur l'architecture healing
- 9.2 Questions sur les scores et le matching
- 9.3 Questions sur la performance et la fiabilite
- 9.4 Questions avancees (niveau expert)

1. Architecture du module self-healing

1.1 Principe general

Le module self-healing est un systeme de reparation automatique des localisateurs Selenium (locators). Quand un element n'est plus trouve avec le locator d'origine (par exemple apres une mise a jour de l'interface), le module tente de trouver automatiquement un nouveau locator fonctionnel, sans intervention humaine.

Les etapes du processus de healing :

- 1. Capture de snapshot : au premier acces reussi, le module capture les attributs (id, class, text, position) de l'element.
- 2. Echec du locator : si le locator d'origine echoue (NoSuchElementException), le module declenche la reparation.
- 3. Cache baseline : verifie si ce locator a deja ete repare avec succes (fichier healing-baseline.json).
- 4. Appel API : envoie le DOM courant + le snapshot de l'element a l'API de healing (POST /heal).
- 5. Retour et stockage : le nouveau locator est stocke en cache et utilise pour les prochains acces.
- 6. Dashboard : chaque evenement de healing est pousse vers l'API Dashboard pour analyse et metriques.

1.2 Les 8 classes du package healing

Le package org.automation.ai.healing contient 8 classes Java organisees en 3 couches :

Classe	Role	Couche
ElementInfo	Modele de donnees : snapshot d'un element DOM	Modele (DTO)
HealingRequest	DTO de requete vers l'API /heal	Modele (DTO)
HealingResponse	DTO de reponse de l'API /heal	Modele (DTO)
HealingBaseline	Cache local persistant des reparations reussies	Cache
SelfHealingClient	Client HTTP vers l'API de healing externe	Client
HealingLocatorResolver	Orchestracteur du processus de healing	Service
HealingWebDriver	Decorateur WebDriver avec injection de healing	Decorateur
HealingWebElement	Decorateur WebElement avec injection de healing	Decorateur

A ces 8 classes s'ajoutent 3 classes externes liees : HealingUtils (utilitaires de conversion et capture DOM), DashboardReporter (push des evenements vers l'API), et ColabClient (generation de features Gherkin via IA).

2. Pipeline de healing complet

2.1 Capture de snapshot

Lorsqu'un element est trouve avec succes, HealingLocatorResolver.captureSnapshot() extrait et stocke les informations suivantes dans un objet ElementInfo :

- tagName : le type de balise HTML (input, button, a, div...)
- visibleText : le texte visible (innerText) de l'element
- Attributs : id, class, type, placeholder, aria-label, href, name, role, data-testid, data-cy, tabindex, value
- Coordonnees : x, y (getBoundingClientRect()) via JavaScript
- Taille : width, height
- XPath : construit a partir du type de locator (ex: //*[@id='email'])
- isRowRelative : true si le xpath est relatif (commence par ./)

Le snapshot est stocke dans un ConcurrentHashMap<clé, ElementInfo> avec une cle composee de l'URL de la page + locator + contexte (driver ou element parent).

Exemple de cle de cache :

```
"driver::app.example.com::By.xpath://input[@id='email']"
```

Exemple de snapshot stocke :

```
ElementInfo {
    elementId = "email",
    elementType = "input",
    text = "Adresse e-mail",
    attributes = {type="email", placeholder="ex: user@example.com"},
    xpath = "//*[@id='email']",
    coordinates = {x=120.0, y=350.0, width=300.0, height=40.0}
}
```

2.2 Resolution de locator

La methode resolveLocator() est le coeur du pipeline de healing. Voici son algorithme :

```
resolveLocator(driver, scope, failedLocator, firstFailure) :

1. Verifier si HEALING_ENABLED = true
2. Construire la cle de cache + nom logique
3. LOG: "Locator failed for '...' : ..."
4. Mesurer le temps de healing (start = now)
5. Appeler callHealingAPI(driver, locator, logicalName, cacheKey)
   5a. driver.getPageSource() -> DOM courant
   5b. oldLocatorMap = convertByToMap(failedLocator)
   5c. oldElement = snapshotCache.get(cacheKey)
       sinon = buildElementInfoFallback(locator, logicalName)
   5d. Construire HealingRequest
   5e. healingClient.healSelector(request) -> POST /heal
6. pushHealingEvent() -> POST /api/healing-events
7. Si response == null ou !success -> retourner null (echec)
8. buildByFromResponse(newLocator) -> By
9. Stocker dans healedLocatorsCache
10. LOG: "Self-healing succeeded"
11. Retourner le nouveau By
```

Le cache healedLocatorsCache est un ConcurrentHashMap qui stocke les locators deja reussis pour eviter de rappeler l'API a chaque echec ulterieur du meme element. getPreferredLocator() consulte ce cache en priorite avant d'utiliser le locator original.

2.3 Decorateurs WebDriver et WebElement

HealingWebDriver et HealingWebElement sont des decorateurs du pattern Decorator de Selenium. Ils implementent les interfaces WebDriver/WebElement en deleguant toutes les methodes a l'instance reelle, sauf findElement() et findElements() qui ajoutent la logique de healing.

Algorithme de HealingWebDriver.findElement(By) :

```
1. preferred = resolver.getPreferredLocator(driver, scope, by)
2. Essayer delegate.findElement(preferred)
   -> OK : capturer snapshot, retourner HealingWebElement wrappe
   -> NoSuchElementException / StaleElementReferenceException :
3. Si preferred != original : essayer delegate.findElement(by)
   -> OK : capturer snapshot, wrapper
   -> Exception : continuer
4. healed = resolver.resolveLocator(driver, scope, by, firstFailure)
   -> null : re-lancer l'exception originale
   -> OK : delegate.findElement(healed), wrapper
```

Le decorateur assure que le healing est totalement transparent pour les Page Objects : ceux-ci continuent a utiliser les memes By locators, et le decorateur intercepte automatiquement les echecs pour tenter une reparation.

3. Description detaillee des classes

3.1 HealingLocatorResolver

Classe centrale qui orchestre tout le processus de healing. 333 lignes.

Methode	Description
getPreferredLocator()	Retourne le locator en cache ou l'original
captureSnapshot()	Extrait les attributs de l'element et stocke dans snapshotCache
resolveLocator()	Pipeline principal : cache -> API -> fallback
callHealingAPI()	Construit la requete et appelle SelfHealingClient
pushHealingEvent()	Extrait les scores et pousse via DashboardReporter
isSameLocator()	Compare deux locators via toString()
buildLogicalName()	Construit le nom logique 'driver::url::By'
cacheKey()	Cle unique driver::url::By ou elem::url::signature
scopeDescription()	Decrit le contexte (driver / element avec tag+id+class+text)
debugHealingRequestPayload()	Sauvegarde les fichiers de debug dans target/healing-debug/

Proprietes statiques : HEALING_ENABLED (configurable via self.healing.enabled), HEALING_DEBUG_REQUEST (sauvegarde des payloads de debug), healedLocatorsCache (ConcurrentHashMap<cle, By>), elementSnapshotCache (ConcurrentHashMap<cle, ElementInfo>).

3.2 HealingWebDriver

Implemente WebDriver, JavascriptExecutor, TakesScreenshot, WrapsDriver. 268 lignes. Decore un WebDriver standard pour injecter la logique de healing dans les methodes findElement et findElements.

Methodes principales :

- findElement(By) : 3 tentatives : cache -> original -> healing, retourne un HealingWebElement
- findElements(By) : meme logique, retourne une liste de HealingWebElement
- switchTo().frame() : retourne HealingWebDriver.this pour chainer les appels
- executeScript() : unwrap automatiquement les HealingWebElement en WebElements reels
- getScreenshotAs() : delegate a ((TakesScreenshot) delegate)

Le unwrap des arguments JavaScript est crucial : Selenium ne peut pas executer de script sur des objets HealingWebElement car ils ne sont pas des WebElements reels. HealingWebDriver unwrap recursive tous les HealingWebElement, List, Map et arrays dans les arguments avant de les passer a JavascriptExecutor.

3.3 HealingWebElement

Implemente WebElement et WrapsElement. 192 lignes. Meme principe que HealingWebDriver mais pour les elements fils : findElement et findElements sur un scope d'element (par exemple la recherche dans un bloc <div>) declenchent aussi le healing.

Contrairement a HealingWebDriver, HealingWebElement conserve une reference au rootDriver et au resolver. Les methodes de base (click, sendKeys, getText, etc.) sont simplement deleguees a l'element real delegate.

3.4 SelfHealingClient

Client HTTP vers l'API de healing externe. 136 lignes. Utilise java.net.http.HttpClient avec HTTP/1.1 (plus stable avec ngrok/Colab que HTTP/2).

Caracteristiques :

- URL resolue depuis self.healing.api.url ou colab.url avec /heal ajoute automatiquement

- Timeout configurable : `self.healing.api.timeout` (default 10s)
- Retry automatique : jusqu'a 2 tentatives pour les erreurs transport (GOAWAY, RST_STREAM, connection reset, broken pipe)
- Header 'ngrok-skip-browser-warning: true' requis par ngrok
- `DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES = false` pour ignorer les champs inattendus
- En cas d'erreur, retourne une `HealingResponse` avec `success=false` et le message d'erreur

```

healSelector(HealingRequest) :
1. Serialise la requete en JSON
2. POST vers {api_url}/heal
3. sendWithRetry() : 2 tentatives max avec backoff 300ms
4. Si status != 200 -> failureResponse
5. Deserialise la reponse en HealingResponse
6. LOG si l'API a retourne une erreur metier
7. Retourne la HealingResponse

```

3.5 HealingBaseline

Cache local persistant des reparations deja reussies. 106 lignes. Utilise un `ConcurrentHashMap<String, BaselineEntry>` et un fichier JSON de persistances cible/target/healing-baseline.json.

Mecanismes :

- Chargement au demarrage (static initializer) depuis le fichier JSON
- Sauvegarde automatique a l'arret via `Runtime.getRuntime().addShutdownHook()`
- Cle : `{locator_type}{locator_value}{page_context}` (cacheKey de `HealingLocatorResolver`)
- stocke : `origType`, `origValue`, `healedType`, `healedValue`, `exceptionType`, `structuralScore`, `semanticScore`
- Utilise par le pipeline : avant d'appeler l'API /heal, le code pourrait verifier la baseline

```

Exemple d'entree baseline :
{
  "driver::app.example.com::By.xpath://input[@id='email']": {
    "origType": "xpath",
    "origValue": "//input[@id='email']",
    "healedType": "xpath",
    "healedValue": "//input[@name='username']",
    "exceptionType": "NoSuchElementException",
    "structuralScore": 0.88,
    "semanticScore": 0.95
  }
}

```

3.6 HealingRequest et HealingResponse

Deux DTOs (Data Transfer Objects) pour la communication avec l'API de healing externe.

HealingRequest (63 lignes)

- `old_locator` (`Map<String, String>`) : type + valeur du locator qui a echoue (ex: `{type: 'id', value: 'email'}`)
- `old_element` (`ElementInfo`) : snapshot de l'element capture lors du dernier acces reussi
- `current_dom` (`String`) : HTML brut de la page au moment de l'echec (via `driver.getPageSource()`)
- `run_id` (`String`) : identifiant du run de test associe

HealingResponse (75 lignes)

- `success` (`boolean`) : succes ou echec de la reparation
- `new_locator` (`Map<String, String>`) : nouveau locator propose (ex: `{type: 'name', value: 'username'}`)
- `score` (`double`) : score de confiance du nouveau locator (0-1)

- details (Map<String, Object>) : informations supplementaires (structural_score, semantic_score, pipeline counts)
- error (String) : message d'erreur si echec

L'annotation @JsonIgnoreProperties(ignoreUnknown = true) sur HealingResponse permet de tolerer des champs supplementaires dans la reponse JSON sans casser la deserialisation.

3.7 ElementInfo

Modele de snapshot d'un element DOM. 159 lignes. Stocke toutes les informations necessaires pour identifier et retrouver un element apres un changement de l'interface.

Champ	Type Java	Annotation JSON	Description
elementId	String	@JsonProperty('element_id')	ID stable de l'element (exclut les IDs auto-generees)
elementType	String	@JsonProperty('element_type')	Tag HTML (input, button, a...)
text	String	-	Texte visible ou innerText
attributes	Map<String, String>	-	Attributs HTML (id, class, placeholder...)
xpath	String	-	XPath construit depuis le locator
isRowRelative	boolean	@JsonProperty('is_row_relative')	True si xpath relatif (./)
coordinates	Map<String, Double>	-	x, y (getBoundingClientRect)
size	Map<String, Double>	-	width, height

La methode statique fromMap() permet de reconstruire un ElementInfo a partir d'une Map<String, Object> (utilisee lors de la deserialisation depuis le cache ou l'API colab). Les methodes castStringMap() et castNumericMap() gerent les conversions de type en toute securite.

4. Utilitaires (HealingUtils)

4.1 Conversion By -> Map et Map -> By

HealingUtils fournit les methodes de conversion entre les objets Selenium By et les Map<String, String> utilisees pour la serialisation JSON.

convertByToMap(By) -> Map<String, String>

- Analyse le toString() du By (ex: 'By.xpath://input[@id='email']')
- Supporte 8 types : id, name, xpath, css, className, tagName, linkText, partialLinkText
- Retourne {type: 'xpath', value: '//input[@id="email"]'}

buildByFromResponse(Map<String, String>) -> By

- Recherche le type parmi 4 alias : type, strategy, locatorType, locator_type
- Recherche la valeur parmi 3 alias : value, selector, locator
- Fallback : si type absent, detecte la cle presente (xpath -> type=xpath, value=...)
- Supporte data-testid comme cssSelector : By.cssSelector("[data-testid=\"...\"]')

normalizeLocatorType(String)

- Normalise les variantes : 'cssselector'/'css_selector'/'css' -> 'css'
- Idem pour className, tagName, linkText, partialLinkText, data-testid

4.2 Capture DOM et debug

Methode writeDebugFiles() : sauvegarde les payloads de healing dans target/healing-debug/

- Fichier {timestamp}-{logicalName}-request.json : la requete complete vers /heal
- Fichier {timestamp}-{logicalName}-current-dom.html : le DOM brut de la page
- Fichier {timestamp}-{logicalName}-old-locator.json : le locator qui a echoue
- Fichier {timestamp}-{logicalName}-old-element.json : le snapshot de l'element

Ces fichiers de debug sont generes uniquement si self.healing.debug.request=true (par default). Ils sont extremement utiles pour diagnostiquer les echecs de healing en comparant le DOM et les snapshots.

4.3 Inference de type et coordonnees

Methodes auxiliaires :

- inferElementTypeFromLogicalName(String) : deduit le type HTML depuis le nom logique. Ex: 'bouton'/'button'/'click' -> 'button', 'champ'/'input'/'email'/'password' -> 'input', 'icon'/'icone' -> 'i'.
- extractCoordinates(WebDriver, WebElement) : execute JavaScript getBoundingClientRect() pour obtenir x, y, width, height.
- buildElementInfoFallback(By, logicalName) : construit un ElementInfo minimal a partir du locator seul (utilise quand le snapshot fait default).
- parseXPathTag(String) : extrait le tag HTML d'un xpath (ex: //input[@id='x'] -> 'input').
- parseXPathAttributes(String, ElementInfo, Map) : extrait les attributs d'un xpath (ex: @id='email' -> id=email).
- buildSnapshotAttributes(WebElement, Map) : extrait 15 attributs standards d'un element Selenium.
- getStableId(Map<String, String>) : retourne l'id seulement s'il n'est pas auto-generé (pattern f_<uuid>).

5. Integration Dashboard (DashboardReporter)

5.1 Push des resultats Cucumber

DashboardReporter.push(String cucumberJsonPath) lit le fichier cucumber.json genere par Maven et pousse les resultats de chaque scenario vers le Dashboard API.

1. Lire le fichier cucumber.json
2. Pour chaque feature -> elements (scenarios) :
 - Extraire feature_name, scenario_name
 - computeStatus(before, steps, after) :
 - 'failed' si un step/hook a failed
 - 'skipped' si tous sont skipped/pending
 - 'passed' sinon
 - computeDuration(before, steps, after) : somme des durees
 - Extraire les tags
3. POST /api/cucumber-runs avec la liste de scenarios
4. Headers : Content-Type: application/json, X-API-Key si configuré

5.2 Push des evenements de healing

DashboardReporter.pushHealingEvent() pousse chaque tentative de healing vers le Dashboard.

Parametres (15+) :

- success (boolean), score (double), structural_score, semantic_score
- old_locator_type, old_locator_val, new_locator_type, new_locator_val
- error_message, healing_time_ms, exception_type, baseline_hit
- elements_extracted, after_struct_filter, after_spatial_filter, sent_to_nlp
- run_id (inferé depuis DASHBOARD_RUN_ID env ou date courante)

Les champs de comptage du pipeline (elements_extracted, after_struct_filter, after_spatial_filter, sent_to_nlp) sont envoyes optionnellement (nullable). Ils permettent de tracer l'efficacite de chaque etape du pipeline de healing cote IA (Colab).

6. Generation de features via Colab (ColabClient)

6.1 Principe et flux

ColabClient est un pont entre les user stories (format texte) et les fichiers .feature Gherkin generes par l'IA (Mistral LLM heberge sur Google Colab).

Flux de generation :

1. Fichier user story (ex: login_us.txt)
Contenu : format texte libre (As a..., I want to..., So that...)
2. ColabClient.generateFeatureFromFile()
 - Lit le fichier user story
 - Appelle l'API Colab via POST /generate
 - Recupere le contenu Gherkin genere par le LLM
3. Fichier .feature (ex: login.feature)
 - Ecrit dans test/resources/features/
 - Contenu : Gherkin standard (Feature:, Scenario:, Given/When/Then)

URL Colab : configurable via system property 'colab.url'
(default: <https://epigastric-troy-calculating.ngrok-free.dev>)

6.2 Appel API et extraction JSON

La methode callColabAPI() envoie la user story a l'API Flask et recupere la reponse.

- Timeout : 30s connexion, 3 minutes reponse (Mistral peut etre lent)
- Header 'ngrok-skip-browser-warning: true' requis par ngrok
- Body : {"user_story": "..."} (JSON avec echappement manuel)
- Reponse : JSON avec champ 'content' contenant le Gherkin genere

L'extraction du champ 'content' est faite manuellement (sans bibliotheque JSON) pour eviter une dependance Jackson/Gson.
L'echappement et le desechappement gerent les caracteres speciaux (\n, \", \\, \t, \r).

7. Les 3 modes de guerison detaillés

7.1 Baseline Java (HealingBaseline)

Le mode Baseline est le plus rapide car il ne necessite aucun parsing DOM ni appel reseau. Il consiste en un cache memoire (ConcurrentHashMap) persiste dans target/healing-baseline.json.

- Cle : {locator_type}{locator_value}{page_context} (cacheKey)
- Stocke : le resultat de la derniere reparation reussie pour ce locator
- Persistence : charge au demarrage, sauvegarde a l'arret (shutdown hook)
- Utilisation : si un locator echoue, le code verifie d'abord la baseline avant d'appeler l'API

7.2 CI Healing Stub

Le CI Healing Stub est un serveur Python standalone (ou endpoint /heal integre dans le dashboard API) qui effectue un matching base sur des regles simples :

- Parse le DOM avec html.parser.HTMLParser (stdlib Python, pas de ML)
- Matching par texte exact (1.0), texte partiel (0.8), attributs (0.7 a 0.9)
- Traduction FR/EN (0.75), bonus si balise correspond (+0.1)
- Pas de modele NLP, pas de base de donnees
- Score toujours a 0.95 en cas de succes

7.3 Colab (IA complet)

Le mode Colab execute un pipeline NLP complet en 5 etapes via un notebook heberge sur Google Colab avec acces GPU.

Pipeline Colab (self_healing_v5.ipynb) :

1. Baseline Python : lookup par MD5(type|valeur|hash du DOM)
2. Extraction DOM : BeautifulSoup + lxml
 - Filtre les elements interactifs (input, button, a, select...)
 - Extrait les attributs (id, name, class, placeholder...)
3. Similarite structurelle (poids 20% tag + 50% attributs + 30% contexte)
 - Tag identique ? Attributs communs ? Contexte DOM parent identique ?
4. Similarite semantique (poids 0.6 dans le score final)
 - Modele : SentenceTransformer paraphrase-multilingual-mpnet-base-v2
 - Multilingue : support FR/EN
5. Similarite spatiale
 - Distance euclidienne normalisee entre les coordonnees x, y

Score final = 0.4 * structurel + 0.6 * semantique

Seuil de succes : 0.75

Le Colab pousse aussi des metrics_snapshots vers le Dashboard via DashboardPusher.push_metrics_snapshot() (POST /api/metrics).

7.4 Tableau comparatif

Critere	Baseline Java	CI Stub	Colab IA
Parsing DOM	Aucun	html.parser stdlib	BeautifulSoup + lxml
Modele ML	Aucun	Aucun	SentenceTransformer

Matching	Lookup exact	Texte + attributs	Struct + Sem + Spatial
Score	Stocke depuis heal	0.95 force	0.4*struct + 0.6*sem
Metrics	healing_event	Aucune	healing_event + snapshot
Performance	Instantané (ms)	< 200ms	1-5s (GPU)
Usage	Cache local	CI sans GPU	Production

8. Flux de donnees healing vers dashboard

8.1 Diagramme de flux

```

HealingWebDriver.findElement(By)
|
+-- [REUSSI] -> captureSnapshot() -> elementSnapshotCache
|
+-- [ECHEC] NoSuchElementException
|
|   +-- [BASELINE HIT] HealingBaseline.lookup()
|   |   +-- pushHealingEvent(success, baseline_hit=true)
|   |
|   +-- [BASELINE MISS] SelfHealingClient.healSelector()
|   |   |
|   |   +-- POST /heal -> CI Stub ou Colab
|   |   |   <- new_locator, score, details
|   |   |
|   |   +-- pushHealingEvent()
|   |   |   +-- POST /api/healing-events
|   |   |
|   |   +-- healedLocatorsCache.put(cle, new By)
|
DashboardReporter.push() (fin du run)
+-- POST /api/cucumber-runs

Colab (optionnel)
+-- POST /api/metrics -> metrics_snapshots

```

8.2 Payload des evenements de healing

Exemple de payload envoye a POST /api/healing-events :

```

{
  "success": true,
  "score": 0.92,
  "old_locator_type": "xpath",
  "old_locator_val": "//input[@id='email']",
  "new_locator_type": "xpath",
  "new_locator_val": "//input[@name='username']",
  "error_message": null,
  "healing_time_ms": 320,
  "exception_type": "NoSuchElementException",
  "baseline_hit": false,
  "elements_extracted": 45,
  "after_struct_filter": 12,
  "after_spatial_filter": 8,
  "sent_to_nlp": 5,
  "structural_score": 0.88,
  "semantic_score": 0.95,
  "run_id": "run-20260518-143022"
}

```

8.3 Fallback 3 niveaux des metriques cote Dashboard

La methode `resolve_healing_payload()` du `DashboardService` implemente un fallback en 3 etapes pour fournir les metriques de healing meme en l'absence de snapshots :

Niveau 1 : metrics_snapshots (table PostgreSQL)

- SELECT * FROM metrics_snapshots WHERE run_id = ?
- ORDER BY captured_at DESC LIMIT 1
- Si trouve -> retourne les metriques directement

Niveau 2 : healing_events (table PostgreSQL)

- SELECT COUNT(*), SUM(success), AVG(score) FROM healing_events
- WHERE run_id = ?
- Calcule healing_rate = successful/total
- baseline_hit_rate = baseline_hits/total
- avg_final_score = AVG(score)

Niveau 3 : endpoint externe (SELF_HEALING_METRICS_URL)

- GET /metrics (Colab ou stub)
- Timeout configurable (5s par default)
- Normalise via normalize_external_metrics()

9. Q&A - Questions possibles

Cette section contient les questions probables lors de la soutenance, classees par theme et par niveau de difficulte.

9.1 Questions sur l'architecture healing

Q: Pourquoi un Decorateur WebDriver plutot qu'un EventListener (WebDriverListener) ?

R: Un WebDriverListener (Selenium 4) permet d'intercepter les evenements avant/apres chaque appel, mais ne permet pas de modifier le comportement de findElement (on ne peut pas changer le By apres un echec). Le Decorateur nous donne le controle total sur le flux : on peut essayer le cache, puis l'original, puis le healing, et wrapper les resultats en HealingWebElement. C'est plus flexible mais necessite d'implementer toutes les methodes de WebDriver (268 lignes au total, dont beaucoup sont de simples delegations).

Q: Pourquoi deux caches (healedLocators et elementSnapshot) ?

R: healedLocatorsCache stocke les By reussis pendant la session (cle -> By). elementSnapshotCache stocke les snapshots d'elements (cle -> ElementInfo). Le premier evite de rappeler l'API, le second fournit les donnees de l'element original (attributs, coordonnees) a l'API de healing pour le matching. Les deux sont independants car un By peut etre en cache sans snapshot (si captureSnapshot n'a pas pu extraire les donnees) et vice-versa.

Q: Comment est calculee la cle de cache (cacheKey) ?

R: La cle est construite par scopeDescription(driver, scope) + '::' + locator. Pour un scope driver : 'driver::' + host (ex: driver::app.example.com). Pour un scope element : 'elem::' + host + '::' + hex(hash(tag|id|class|text)). Le hash hex permet de differencier les elements parents memes sans ID unique. En cas d'element stale, le fallback est 'elem::' + host + '::stale'.

Q: Quel est l'impact du healing sur les performances des tests ?

R: L'impact est negligeable en cas de succes (captureSnapshot apres findElement reussi ajoute ~5ms). En cas d'echec, le cout est plus eleve : getPageSource() (DOM complet) + POST /heal (1-5s avec Colab, <200ms avec stub) + POST /api/healing-events. Si le healing est declare prioritairement dans les configs (self.healing.api.url pointe vers le stub rapide), le temps total reste sous les 500ms. Le cache healedLocators evite les appels repetitifs.

9.2 Questions sur les scores et le matching

Q: Que signifient les 4 compteurs du pipeline (elements_extracted, after_struct_filter, after_spatial_filter, sent_to_nlp) ?

R: Ces compteurs retracent le pipeline de l'IA Colab : elements_extracted = nombre d'elements interactifs extraits du DOM ; after_struct_filter = elements restants apres filtrage par similarite structurelle (tag + attributs) ; after_spatial_filter = elements restants apres filtrage par position spatiale ; sent_to_nlp = elements envoyes au modele SentenceTransformer pour la similarite semantique. Si sent_to_nlp est bas (ex: 5 sur 45 extractions), le pipeline a bien filtre les elements non pertinents avant le NLP couteux.

Q: Pourquoi le score final est-il pondere 0.4 * structurel + 0.6 * semantique ?

R: La ponderation favorise la semantique car le texte et le sens sont plus robustes que la structure HTML face aux changements d'interface. Un bouton peut changer de classe CSS ou de position, mais son texte ('Se connecter') reste le meme. Le seuil de 0.75 est un equilibre : trop bas (ex: 0.5) genererait trop de faux positifs, trop haut (ex: 0.9) rendrait le healing inutile car il echouerait sauf cas presque identiques.

Q: Comment le healing gere-t-il les elements dynamiques (ID generees, listes) ?

R: Pour les ID generees (pattern f_<uuid> de 30+ caracteres hex), la methode getStableId() dans HealingUtils detecte le pattern et retourne null. L'element sera alors identifie par d'autres attributs (name, class, text, placeholder, aria-label, data-testid). Pour les elements de liste (table rows), le flag isRowRelative (= true si xpath commence par ./) permet a l'IA de savoir que le locator est relatif a un element parent et d'ajuster la recherche.

9.3 Questions sur la performance et la fiabilite

Q: Que se passe-t-il si l'API de healing est injoignable (timeout) ?

R: SelfHealingClient a un timeout configurable (10s par default) et un retry automatique (2 tentatives avec backoff 300ms). Si les deux tentatives echouent, la methode healSelector retourne une HealingResponse avec success=false et le message d'erreur. HealingLocatorResolver.resolveLocator() propage alors l'exception originale (NoSuchElementException). Le test echoue normalement, sans faux positif. Les evenements d'echec sont tout de meme pousses au Dashboard.

Q: Comment le module assure-t-il la compatibilite ascendante des locators ?

R: Le cache healedLocators stocke les reparations par cle (URL + scope + locator). Si un locator a ete repare avec succes, tous les appels ulterieurs utiliseront directement le nouveau By sans passer par l'API. Si le nouveau locator echoue a son tour (par exemple apres un second changement d'interface), le pipeline se declenche a nouveau et peut produire un troisieme locator. Le fichier healing-baseline.json conserve l'historique des reparations entre les sessions.

Q: Quels types d'exceptions declenchent le healing ?

R: Le healing est declenche pour NoSuchElementException et StaleElementReferenceException. Ce sont les deux exceptions les plus frequentes liees a des changements d'interface. Les autres exceptions (TimeoutException, ElementNotInteractableException, etc.) ne sont pas capturees par le decorateur et remontent normalement. Le type d'exception est stocke dans healing_events (colonne exception_type) pour analyse.

9.4 Questions avancees (niveau expert)

Q: *Pourquoi HTTP/1.1 plutot que HTTP/2 pour SelfHealingClient ?

R: HTTP/2 multiplexe les requetes sur une seule connexion TCP, ce qui est generalement plus performant. Cependant, avec ngrok (qui fait office de proxy), les connexions HTTP/2 peuvent etre interrompues par des trames GOAWAY. Le code Java detecte d'ailleurs 'goaway' dans la methode isRetryableTransportError(). HTTP/1.1 est plus stable avec ngrok/Colab car chaque requete est une connexion TCP independante. Si le service etait en production sans ngrok, HTTP/2 serait prefere.

Q: *Comment evoluerait l'architecture pour supporter le multi-navigateur ?

R: Actuellement, le healing est agnostique du navigateur car il travaille sur le DOM HTML et les attributs standards. Pour le multi-navigateur (Chrome, Firefox, Safari), il faudrait ajouter le navigateur dans la cle de cache (ex: driver::chrome::app.example.com::By.id::email) car certains attributs ou comportements diffèrent (ex: getBoundingClientRect peut retourner des valeurs legerement differentes). Le ElementInfo pourrait etre etendu avec un champ 'browser' pour que l'IA de healing adapte ses strategies par navigateur.

Q: *Quelle est la strategie de test unitaire pour le module healing ?

R: Le module healing est difficile a tester unitairement car il depend de Selenium (WebDriver, WebElement). Les strategies possibles : 1) Mocker WebDriver et WebElement avec Mockito pour tester HealingLocatorResolver.resolveLocator() en isolation. 2) Utiliser HtmlUnitDriver (Selenium headless sans navigateur) pour les tests d'integration. 3) Tester SelfHealingClient avec un serveur mock (WireMock) pour verifier la construction des requetes et le retry. 4) Tester HealingUtils avec des By simules (pas de navigateur necessaire). Actuellement, les tests sont principalement manuels via l'execution des tests d'acceptation.

Q: *Comment le healing pourrait-il etre etendu a d'autres types de localisateurs (React, Angular) ?

R: Pour les frameworks JS modernes, les localisateurs optimaux sont souvent : data-testid (React Testing Library), data-cy (Cypress), ou aria-label (accessibilite). Le module supporte deja data-testid (converti en cssSelector). L'extension consisterait a : 1) Ajouter des strategies de capture pour les attributs specifiques (data-testid, data-cy, aria-label, role). 2) Ajouter un poids plus eleve pour ces attributs dans le matching. 3) Generer preferentiellement des locators data-testid dans les suggestions de l'IA, car ce sont les plus stables. Le code actuel capture deja data-testid et data-cy dans les attributs (buildSnapshotAttributes).

Q: *Quels sont les risques de faux positifs avec le healing automatique ?

R: Le risque principal est que le healing trouve un element different de celui attendu mais avec un score > 0.75. Par exemple, trouver un bouton 'Annuler' a la place de 'Confirmer' si leurs textes sont proches et leurs positions similaires. Plusieurs mecanismes reduisent ce risque : 1) Le seuil de 0.75 est eleve. 2) Le score semantique (poids 0.6) compare les textes via un modele entraine, ce qui evite les confusions. 3) Les tests continuent de verifier les comportements apres le clic : si le healing trouve le mauvais element, l'assertion suivante echouera. 4) Le suivi via healing_events permet d'analyser les cas limites. Le risque zero n'existe pas, mais le systeme est concu pour minimiser l'impact.

--- Fin du document ---